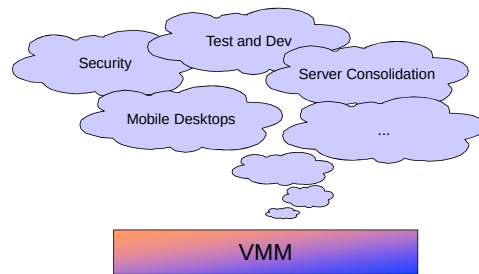




A Comparison of Software and Hardware Techniques for x86 Virtualization

Keith Adams
Ole Agesen
Oct. 23, 2006

VMs are everywhere



X86 virtualization

1998-2005: Software-only VMMs

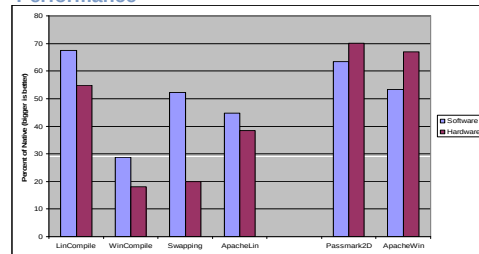
- X86 does not support traditional virtualization
- Binary translation!
- VMMs from VMware, Microsoft, Parallels, QEMU

2005- : Hardware support emerges

- AMD, Intel extend x86 to directly support virtualization
- Direct comparisons now possible!



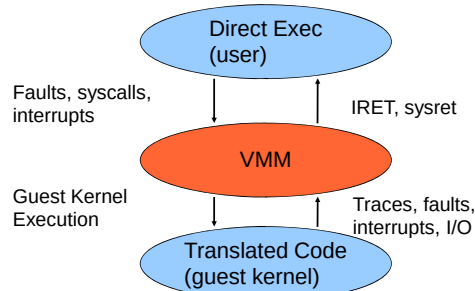
Software vs. Hardware: Performance



- Intuition: hardware is fast!
- A mixed bag; why?



Software VMM



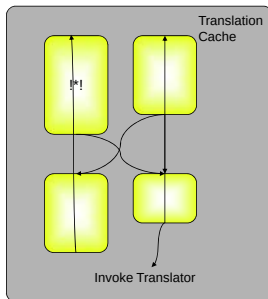
Binary Translation in Action

BT offers many advantages

- Correctness: Non-virtualizable x86 instructions
- Flexibility
 - Guest idle loops, spin locks, etc.
 - Work around guest bugs
 - Transparently instrument guest
- Adaptation
 - *Traces*: VMM write protects guest privileged data, e.g., page tables
 - *Trace faults*: Guest writes page tables -> major source of overhead



Adaptation example



Translation Cache

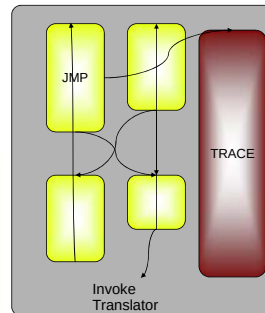
- Captures working set of guest
- Amortizes translation overheads

CFG of a simple guest

High rate of trace faults at instruction '!!'

"Trap-and-emulate" approach => 1000's of CPU cycles

Adaptation example (2)

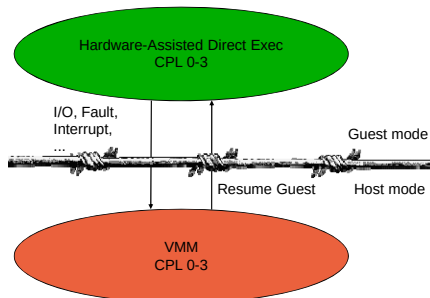


BT Engine splices in special 'TRACE' translation

Executes memory access "in software"

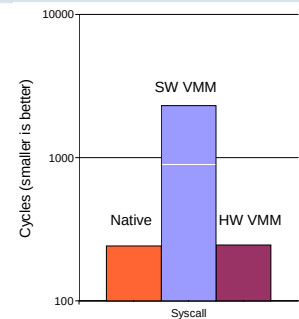
10x improvement in trace performance

Hardware-assisted VMM

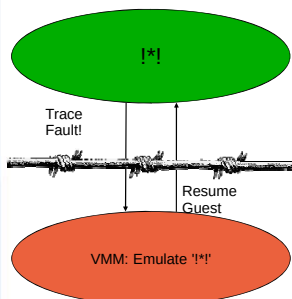


Hardware: System Calls Are Fast

- CPL transitions don't require VMM intervention
- Native speed system calls!



Hardware VMM Trace Faults



Trace fault from '!!'

- Exit from guest mode
- Emulate faulting instruction
- Resume
- Many 1000's of cycles round-trip

VMM notices high rate of faults at '!!', and ...

- does what?

Paetable modification

Native

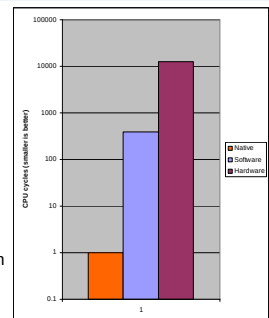
- Simple store
- 1 cycle (!)

Software VMM

- Converges on 'TRACE' translation
- ~400 cycles

Hardware VMM

- No translation -> no adaptation
- ~11000 cycles

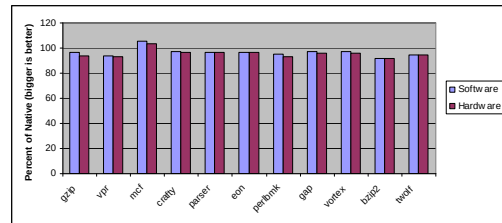


Benchmarks

System under test

- ▶ Pentium 4 672, 3.8 GHz, VT-x
 - ▶ Software VMM: VMware Player 1.0.1
 - ▶ Hardware VMM: VMware Player 1.0.1 (same!)
- <http://www.vmware.com/products/player/>

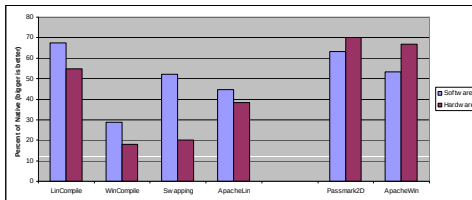
Computation Is a Toss-Up



Direct execution

Both VMMs close to each other, native

Kernel-Intensive Workloads



Some workloads favor hardware, others software

- ▶ Why?
- ▶ Which one should you use?

Nano-benchmarks

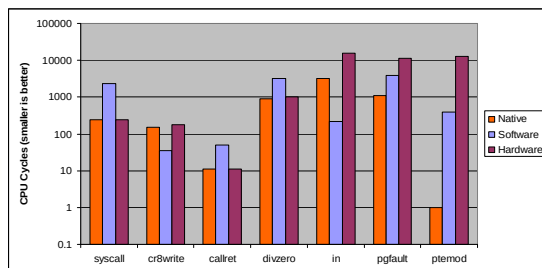
More "micro-" than "micro-"

Measure a single virtualization-sensitive op

- ▶ Often a single instruction

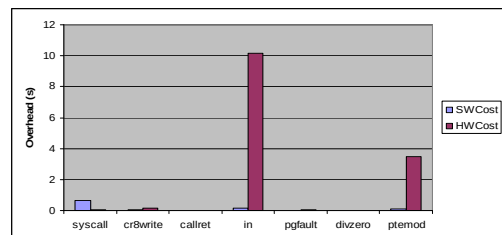
Nano-bench results + workload's mix of virtualization ops => crude performance prediction

Nano-Benchmark Results



- Software: wins some (even vs. native), loses some
- Hardware: bimodal: native speed or ~11000 cycles(!)

Decomposing a Macro-Benchmark: XP64 Boot/Halt



- Estimated overhead = Frequency * nanobench score
- "in" overhead is anomalous (boot-time BIOS initialization code)

Two Workloads That Favor Hardware

Passmark/2D

- I/O to weird device: no VMM intervention

Apache/Windows

- Performing many thread switches
- No exits on hardware VMM
- ...but "purpose" of Apache is I/O, not thread switches

They are system call micro-benchmarks in disguise

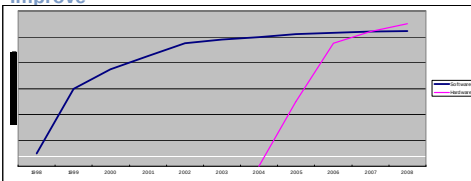
Claim: These two workloads are anomalous.

Which VMM Should I Use?

"It depends."

- Computation: flip a coin
- "Trivial" kernel-intensive
 - Single address space, little I/O
 - => Hardware!
- "Non-trivial" kernel-intensive
 - Process switches, I/O, address space modifications
 - => Software!!!

Claim: Hardware Will Improve



Micro-architecture: faster exits, VMCB accesses, ...

Architecture: assists for MMU, APIC, more decoding, ...

Software: tuning, maturity, ...

Conclusions

Current hardware does not achieve performance-parity with previous software techniques.

Major problem for accelerating virtualization

- Not executing the virtual instruction stream...
- But efficient virtualization of MMU and I/O

Hardware should enhance, not replace, software techniques

Backup slides

Improving Virtual MMU Performance

Tune existing software MMU

- Inherited from SW VMM
- Can use traces more lightly, but...
- Trade performance in other parts of the system

Current hardware introduces new constraints

- Fundamentally harder for software MMU

Hardware approach

- Intel's "EPT", AMD's "NPT"
- Hardware walks 2nd level of page tables on TLB misses

WinXP64 Boot/Halt Translation Stats

t (in 10s)	units	size	instr	cycles	size	cyc/ins	ins/unit
0	38690	336k	120k	252M	924k	2097	3.11
1	48839	500k	169k	318M	1164k	1871	3.48
2	108k	1187k	392k	754M	2589k	1920	3.61
3	29362	264k	89749	287M	951k	3197	3.06
4	96876	1000k	337k	708M	2418k	2100	3.48
5	58553	577k	193k	403M	1572k	2078	3.31
6	19430	148k	50951	148M	633k	2904	2.62
7	13081	87811	30455	124M	494k	4071	2.33
Total	413k	4101k	1384k	2994M	10748k	2161	3.35